

LMDB :

la base NoSQL la plus rapide du monde

OpenLDAP-LMDB ou LMDB est une base de données embarquée transactionnelle de stockage clé-valeur disponible sous Linux. En 2009 le projet OpenLDAP est préoccupé par l'avenir de Berkeley DB passé dans le giron d'Oracle et il est décidé de faire un fork de Berkeley DB. Appelée MDB, la librairie devient LMDB. La librairie est utilisée massivement sous Linux.

niveau
200

Portage sous Windows

Avant de pouvoir l'utiliser sous forme de DLL Windows, il faut migrer le code Linux vers Windows. La librairie contient du code pour Windows mais le portage est incomplet. L'utilisation des noms de fichiers contient un / au lieu de \ qui fait que cela ne marche que sous Linux... Une fois cette modification effectuée, il faut exporter les diverses fonctions pour obtenir une DLL.

LMDBWindowsDll

Le portage de la DLL sous Windows est disponible sur : <https://github.com/ChristophePichaud/LMDBWindows/LMDBWindowsDll>
En Debug, la DLL LMDBWindowsDll64.dll fait 235 KB. En Release, elle en fait moins de 100 KB. C'est ultra-léger.

Utilisation de LMDB API

L'API est du C pur et dur. Il y a des structures à définir et à utiliser. Les fonctions sont préfixées par mdb_. L'utilisation du C est basique mais peut rebuter certaines personnes. Il faut passer des buffers, des longueurs, du C quoi ! Pour être élégant, il vaut mieux encapsuler l'API C en C++ et mettre à disposition des std::string.

```
class LMDBWRAPPER_API CLMDBWrapperEx
{
public:
    CLMDBWrapperEx() {}
    ~CLMDBWrapperEx();

private:
    MDB_env * env;
    MDB_dbi dbi;
    MDB_val val;
    MDB_txn * txn;
    MDB_cursor * cursor;

public:
    void Init(const std::wstring& db)
    void BeginTransaction();
    void CommitTransaction();
    void Set(const std::string& k, const std::string& v)
    bool Get(const std::string& k, std::string& value)
};
```

Cette version d'API est beaucoup plus simple à utiliser. On va déclarer un objet, faire appel à Init en y passant le nom de la base de données, et ensuite on peut faire Get/Set comme on veut...



Christophe PICHAUD
Architecte Microsoft chez
'M Applications by Devoteam'
christophepichaud@hotmail.com
www.windowscpp.com

```
std::string db = "mycache";

CLMDBWrapperEx we;
we.Init(db);
we.BeginTransaction();
std::string key = "key_v000";
std::string value = "value_v000";
we.Set(key, value);

std::string value2;
we.Get(key, value2);
we.CommitTransaction();
```

Comme vous pouvez le constater, l'ajout de données se fait par clé et valeur.

La base de données

Elle est constituée de deux fichiers :

- data.mdb
- lock.mdb

La méthode Init crée un répertoire du nom de la base dans c:\temp pour y stocker ces deux fichiers.

L'ouverture de la base

On vient de voir que Init crée un répertoire mais ce n'est pas tout :

```
void Init(const std::string& db)
{
    char sz[255];
    sprintf_s(sz, "%s\\%s", Constants::LMDBRootPath.c_str(), db.c_str());
    ::CreateDirectoryA(sz, NULL);

    mdb_env_create(&env);
    mdb_env_set_maxreaders(env, 1);
    mdb_env_set_mapsize(env, 10485760 * 100);
    mdb_env_open(env, sz, MDB_CREATE, 0);

    mdb_txn_begin(env, NULL, 0, &txn);
    mdb_dbi_open(txn, NULL, 0, &dbi);
    mdb_txn_commit(txn);
}
```

Init ouvre aussi une connexion à la base via mdb_dbi_open et positionne certains paramètres.

Gestion de la transaction

La transaction est créée via `mdb_txn_begin` et `mdb_txn_commit` :

```
void BeginTransaction()
{
    int err = 0;
    err = mdb_txn_begin(env, NULL, 0, &txn);
}

void CommitTransaction()
{
    int err = 0;
    err = mdb_txn_commit(txn);
}
```

Insertion de données

La création des données se fait via un appel à `mdb_put` :

```
void Set(const std::string& k, const std::string& v)
{
    int err = 0;

    key.mv_size = k.length() + 1;
    key.mv_data = (void *)k.c_str();
    data.mv_size = v.length() + 1;
    data.mv_data = (void *)v.c_str();
    err = mdb_put(txn, dbi, &key, &data, 0); // MDB_NOOVERWRITE;
    //printf("Set err:%d Key:%s Data:%s\n", err, key.mv_data, data.mv_data);
}
```

Récupération de données

La récupération des données se fait via un appel à `mdb_get` :

```
bool Get(const std::string& k, std::string &value)
{
    int err = 0;

    key.mv_size = k.length() + 1;
    key.mv_data = (void *)k.c_str();

    err = mdb_get(txn, dbi, &key, &data);

    if (err != 0)
        return false;

    //printf("Get err:%d Key:%s Data:%s\n", err, key.mv_data, data.mv_data);
    value = (char *)data.mv_data;

    return err == 0 ? true : false;
}
```

Evaluation des performances

Reprenons notre exemple et mettons-lui des indicateurs de temps :

```
int count = 2;

if (argc == 2)
{
    count = atoi(argv[1]);
}
```

```
CLMDBWrapperEx wr1;
wr1.Init(db);

DWORD dwStart = 0;
DWORD dwEnd = 0;

dwStart = ::GetTickCount();
wr1.BeginTransaction();
for (int i = 0; i < count; i++)
{
    std::string k = "key_v" + std::to_string(i);
    std::string v = "value_v" + std::to_string(i);
    wr1.Set(k, v);
}
wr1.CommitTransaction();
dwEnd = ::GetTickCount();
std::cout << "Elapsed Time for SET: "
<< dwEnd - dwStart << " ms" << std::endl;

dwStart = ::GetTickCount();
wr1.BeginTransaction();
for (int i = 0; i < count; i++)
{
    std::string k = "key_v" + std::to_string(i);
    std::string v;
    wr1.Get(k, v);
}
wr1.CommitTransaction();
dwEnd = ::GetTickCount();
std::cout << "Elapsed Time for GET: "
<< dwEnd - dwStart << " ms" << std::endl;
```

Le programme est simple. Il prend un nombre d'opérations à réaliser et ensuite il fait autant de GET et autant de SET sur un jeu de clé / valeur simple. Testons ce petit programme :

Opération	Nombre	Durée (ms)
get	1	0
set	1	0
get	10	0
set	10	0
get	100	15
set	100	0
get	1000	63
set	1000	15
get	10000	109
set	10000	63
get	100000	1141
set	100000	640

Modifions le programme pour insérer et récupérer des documents JSON :

```
std::string jsonDocument = R("User": {
    "Account": "%s@agenda.com",
    "Password": "agenda",
    "AdjustTime": 1,
    "FiggioURL": "https://wendelgroup.ilucca.net/",
    "FiggioToken": "aaaaaaa-fb8f-41a3-a08e-84b6521ec96a",
    "EnableFiggio": false
});
```

```

dwStart = ::GetTickCount();
wr1.BeginTransaction();
for (int i = 0; i < count; i++)
{
    char sz[1024];
    std::string k = "key_j" + std::to_string(i);
    sprintf_s(sz, jsonDocument.c_str(),
std::to_string(i).c_str());
    std::string v = sz;
    wr1.Set(k, v);
}
wr1.CommitTransaction();
dwEnd = ::GetTickCount();
std::cout << "Elapsed Time for SET jSON: "
<< dwEnd - dwStart << " ms" << std::endl;

dwStart = ::GetTickCount();
wr1.BeginTransaction();
for (int i = 0; i < count; i++)
{
    std::string k = "key_v" + std::to_string(i);
    std::string v;
    wr1.Get(k, v);
}
wr1.CommitTransaction();
dwEnd = ::GetTickCount();
std::cout << "Elapsed Time for GET jSON: "
<< dwEnd - dwStart << " ms" << std::endl;

```

Voici le résultat :

Opération	Nombre	Durée (ms)
get jSON	1	0
set jSON	1	0
get jSON	10	0
set jSON	10	0
get jSON	100	0
set jSON	100	0
get jSON	1000	16
set jSON	1000	16
get jSON	10000	218
set jSON	10000	63
get jSON	100000	2375
set jSON	100000	672

Les performances sont exceptionnelles. La taille de la base de données est conséquente (53 MB) :

```

D:\Dev\LMDBWindows\LMDBWindows\x64\Debug>dir c:\temp\mycache
Directory of c:\temp\mycache
02/10/2019 05:11 AM <DIR> .
02/10/2019 05:11 AM <DIR> ..
02/10/2019 05:11 AM    53,145,600 data.mdb
02/10/2019 05:11 AM      128 lock.mdb

```

Application pratique

Vous souhaitez utiliser une base de données de cache dans votre site web sur Azure et vous hésitez entre DocumentDB ou

MongoDB. Arrêtez de payer ce genre de services, essayez LMDB, c'est gratuit et beaucoup plus rapide. De plus LMDB possède aussi des fonctionnalités de curseur pour lire le contenu de la base...

Explorer la base

Voici le code qui permet de parcourir les données :

```

void BeginTransactionReadOnly()
{
    int err = 0;

    err = mdb_txn_begin(env, NULL, MDB_RDONLY, &txn);
}

void OpenCursor()
{
    int err;
    err = mdb_cursor_open(txn, dbi, &cursor);
}

void CloseCursor()
{
    mdb_cursor_close(cursor);
}

bool GetFromCursor(std::string & k, std::string & v)
{
    int err = 0;

    err = mdb_cursor_get(cursor, &key, &data, MDB_NEXT);

    if (err != 0)
        return false;

    //printf("Get err:%d Key:%s Data:%s\n",
err, key.mv_data, data.mv_data);

    k = (char*)(key.mv_data);
    v = (char*)(data.mv_data);

    return err == 0 ? true : false;
}

void GetAllData()
{
    int err;
    BeginTransactionReadOnly();

    OpenCursor();

    std::string k, value;
    while (GetFromCursor(k, value))
    {
        printf("key: '%s', value: '%s'\n",
k.c_str(), value.c_str());
    }
}

```

```

    }

    CloseCursor();

    AbortTransaction();
}

```

Le code de GetAllData() n'est pas très compliqué. On déclare un curseur et on itère sur la méthode GetFromCursor. Voici le programme client :

```

CLMDBWrapperEx wr1;
wr1.Init(db);
wr1.GetAllData();

```

Voici la sortie console, on y remarque les documents JSON : 1

Si on modifie le programme client pour avoir des indications de temps :

```
Elapsed Time for GetAllData: 32 ms
```

Il faut moins de 100ms pour charger 200.000 items soit 53 MB de data en RAM.

C'est du C/C++ ! C'est performant. Ultra-performant.

Interop en C# .NET

Il existe des bindings pour le langage C#. C'est le rôle de Lmdb.NET. Un portage C# qui fait des appels Interop à la DLL qui va bien. Voici le diagramme de classes simplifié : 2

Voici comment ouvrir une base :

```

string dir = "c:\\temp\\mycache";
LmdbEnvironment _env = new LmdbEnvironment(dir);
_env.MaxDatabases = 2;
_env.MapSize = 10485760 * 100;
_env.Open();

```

Voici comment faire un Put :

```

var tx = _env.BeginTransaction();
var db = tx.OpenDatabase(null,
new DatabaseConfiguration { Flags = DatabaseOpenFlags.Create });
tx.Put(db, "hello", "world");
tx.Commit();

```

Voici comment faire un Get :

```

tx = _env.BeginTransaction(TransactionBeginFlags.ReadOnly);
db = tx.OpenDatabase(null);
var result = tx.Get(db, "hello");
tx.Commit();
db.Dispose();

```

Voici le code de récupération des informations JSON via le curseur :

```

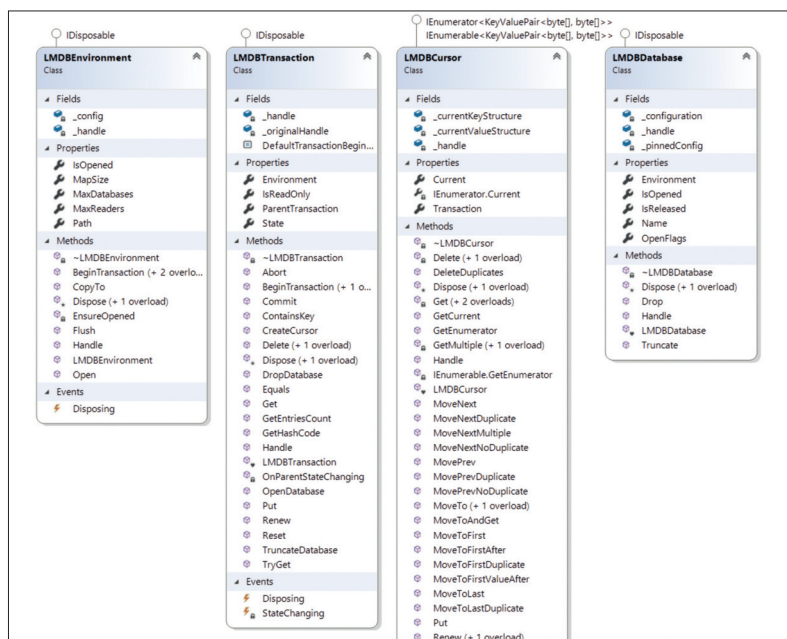
LmdbTransaction tx3 = _env.BeginTransaction(TransactionBeginFlags.NoSync);
LmdbDatabase db3 = tx3.OpenDatabase(null,
new DatabaseConfiguration { Flags = DatabaseOpenFlags.Create });
var cur = tx3.CreateCursor(db3);
while (cur.MoveNext())
{
    var fKey = Encoding.UTF8.GetString(cur.Current.Key);

```

```

D:\Dev\LmdbWindows\LmdbWindows\x64\Debug\LmdbWrapper_Client.exe [more
key: 'key_j0', value: "User": {
  "Account": "100agenda.com",
  "Password": "agenda",
  "AdjustTime": 1,
  "FiggioURL": "https://wendelgroup.ilucca.net/",
  "FiggioToken": "aaaaaaaa-fb8f-41a3-a08e-84b6521ec96a",
  "EnableFiggio": false
},
key: 'key_j1', value: "User": {
  "Account": "100agenda.com",
  "Password": "agenda",
  "AdjustTime": 1,
  "FiggioURL": "https://wendelgroup.ilucca.net/",
  "FiggioToken": "aaaaaaaa-fb8f-41a3-a08e-84b6521ec96a",
  "EnableFiggio": false
},
key: 'key_j10', value: "User": {
  "Account": "100agenda.com",
  "Password": "agenda",
  "AdjustTime": 1,
  "FiggioURL": "https://wendelgroup.ilucca.net/",
  "FiggioToken": "aaaaaaaa-fb8f-41a3-a08e-84b6521ec96a",
  "EnableFiggio": false
},
key: 'key_j100', value: "User": {
  "Account": "100agenda.com",
  "Password": "agenda",
  "AdjustTime": 1,
  "FiggioURL": "https://wendelgroup.ilucca.net/",
  "FiggioToken": "aaaaaaaa-fb8f-41a3-a08e-84b6521ec96a",
  "EnableFiggio": false
}
-- More --

```



```

var fValue = Encoding.UTF8.GetString(cur.Current.Value);
string str3 = String.Format("key:{0} => value:{1}", fKey, fValue);
Logger.LogInfo(str3);
}
tx3.Commit();
db3.Dispose();
_env.Dispose();

```

Conclusion

Que vous souhaitiez utiliser SQLite ou un fichier de config, utiliser une base Azure comme Document DB ou MongoDB, il faut maintenant prendre en compte cette nouvelle librairie qu'est Lmdb. C'est gratuit, rapide et facile à mettre en œuvre. Elle expose les performances de toutes les bases NoSQL. Vous pouvez l'utiliser dans un container Docker avec la mise à disposition des données, soit volatiles dans le container, soit persistantes dans des volumes partagés. Lmdb et Lmdb.NET, c'est le meilleur des deux mondes, le C++ pour la rapidité et le C# pour la facilité d'utilisation. Lancez-vous !